Reporte de resultados: Botón de arranque/paro a prueba de rebotes y apagones

Práctica 4.1.2 · Sistemas Programables

1. Objetivo de las pruebas

Comprobar que el sistema arranca y detiene el motor con un solo botón (una pulsación lo arranca y la siguiente lo apaga) y que se defiende de los dos fenómenos físicos de la práctica: el rebote del botón, con el filtro no bloqueante de la clase `Boton`, y los reinicios por caída de voltaje, con un arranque seguro en el que el motor nunca se mueve sin una orden nueva. También se verifica que la rampa de PWM lleve el motor de 0 a 255 en 1.5 s y que el paro sea inmediato.

2. Condiciones de prueba

- Pruebas hechas en Tinkercad Circuits, porque el equipo no contaba con el Arduino físico. Tinkercad no tiene el UNO R4 WiFi ni el L298N, así que se usó un Arduino UNO R3 con el chip L293D (otro puente H que se controla igual), un motor de CC con pila de 9 V y una LCD 16x2 I2C en lugar de la matriz LED.
- Conexiones: D9 (PWM) a EN1,2 del L293D, D7 a IN1, IN2 a GND; pulsador entre D2 y GND con `INPUT_PULLUP`.
- Parámetros del programa: ventana del filtro de 25 ms, rampa de 0 a 255 en 1500 ms, aviso de reinicio de 3 s.
- Monitor Serie a 115200 baudios, con los comandos `V n` (cambia la ventana del filtro), `C` (contador a 0) y `G` (alterna el arranque entre rampa y golpe), para no volver a subir el programa en cada prueba.
- El apagón se simuló deteniendo e iniciando de nuevo la simulación con el motor girando.
- Práctica realizada en equipo.

3. Datos registrados

3.1 Resultado de cada prueba

La tabla resume lo que se observó en la LCD, en el motor y en el Monitor Serie durante cada prueba.

#	Prueba	Resultado observado	¿Cumple?
1	Arranque seguro	La LCD mostró «*** SISTEMA *** REINICIADO», el Monitor Serie imprimió `*** SISTEMA REINICIADO ***` y el motor quedó en 0 rpm. A los 3 s la LCD cambió a «Motor: OFF».	Sí

#	Prueba	Resultado observado	¿Cumple?
2	Arranque con rampa	Con la primera pulsación la LCD mostró «Motor: RAMPA» y la barra fue subiendo. Al terminar se imprimió `Rampa terminada: motor a velocidad maxima (PWM 255)` y la LCD pasó a «Motor: ON».	Sí
3	Paro	Con la siguiente pulsación el motor se detuvo de inmediato y la LCD volvió a «Motor: OFF».	Sí
4	Apagón	Con el motor girando se detuvo y reinició la simulación. El sistema volvió apagado, con el aviso de reinicio, y el contador empezó en `Pulsacion #1` solo al oprimir el botón otra vez.	Sí
5	Rampa contra golpe	Con `G` el motor pasó directo a velocidad máxima («Motor: ON» sin pasar por «RAMPA»).	Sí
6	Ventana del filtro	Con `V 300` una pulsación corta no se contó; con `V 25` la misma pulsación corta sí se contó (`Pulsacion #4`).	Sí
7	Rebote (10 pulsaciones)	No se pudo medir: el pulsador de Tinkercad es ideal y no rebota (ver sección 5).	Pendiente

3.2 Líneas del Monitor Serie

Transcripción de las líneas visibles en las dos capturas, en el orden en que aparecen. La pulsación corta ignorada con la ventana de 300 ms no genera ninguna línea, porque el filtro nunca la aceptó.

Captura	Línea impresa	Qué indica
Fig. 1	`Rampa terminada: motor a velocidad maxima (PWM 255)`	El motor terminó la rampa antes del apagón.
Fig. 1	`*** SISTEMA REINICIADO ***`	Aviso de reinicio después del apagón.
Fig. 1	`Motor APAGADO (estado seguro). Oprima el boton para arrancar.`	El motor vuelve apagado.
Fig. 1	`Ventana del filtro: 25 ms`	Parámetros con los que arranca.
Fig. 2	`Pulsacion #2 -> Motor APAGADO`	Paro con la segunda pulsación.
Fig. 2	`Arranque: con rampa (1.5 s)`	Comando G: se regresa al arranque con rampa.
Fig. 2	`Ventana del filtro: 300 ms`	Comando V 300.
Fig. 2	`Pulsacion #3 -> Motor ARRANCANDO (rampa)`	Una pulsación normal sí pasa con 300 ms.
Fig. 2	`Ventana del filtro: 25 ms`	Comando V 25.
Fig. 2	`Pulsacion #4 -> Motor APAGADO`	La pulsación corta ya se cuenta con 25 ms.

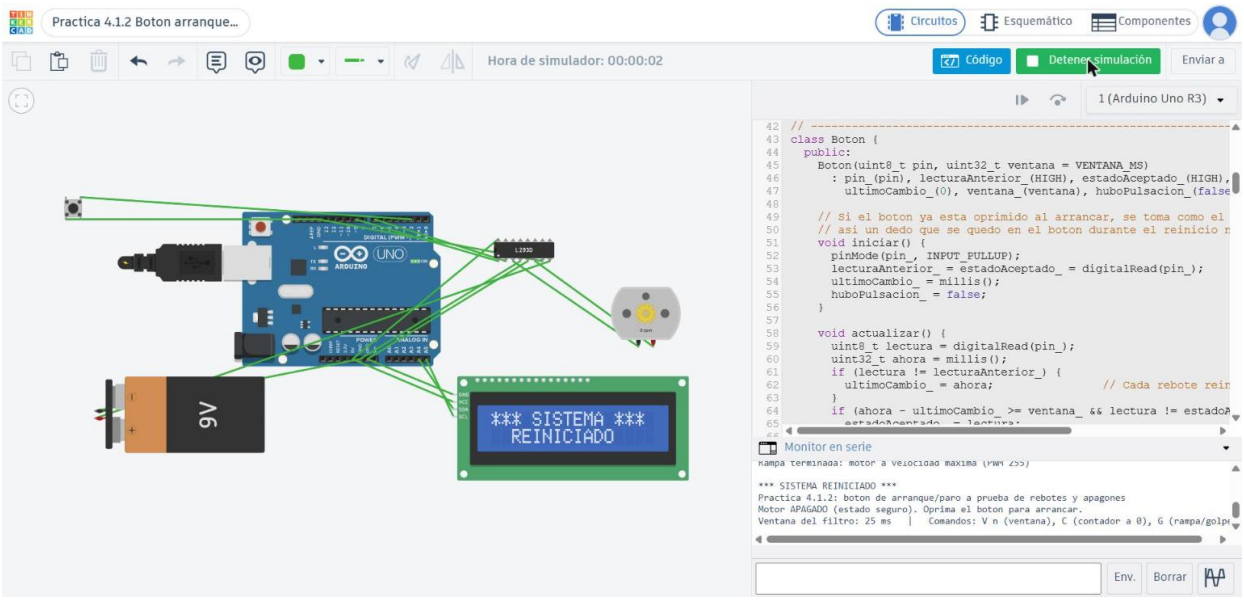


Figura 1. Arranque en estado seguro después del apagón: aviso en la LCD y en el Monitor Serie, motor en 0 rpm.

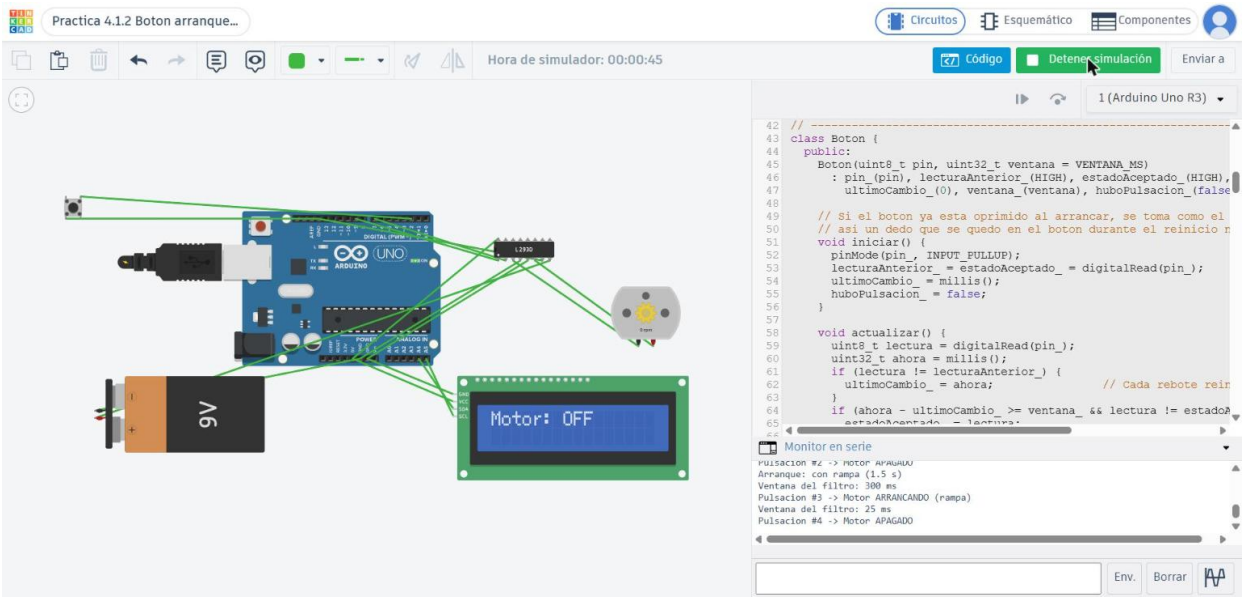


Figura 2. Monitor Serie con la prueba de la ventana del filtro (300 ms y 25 ms).

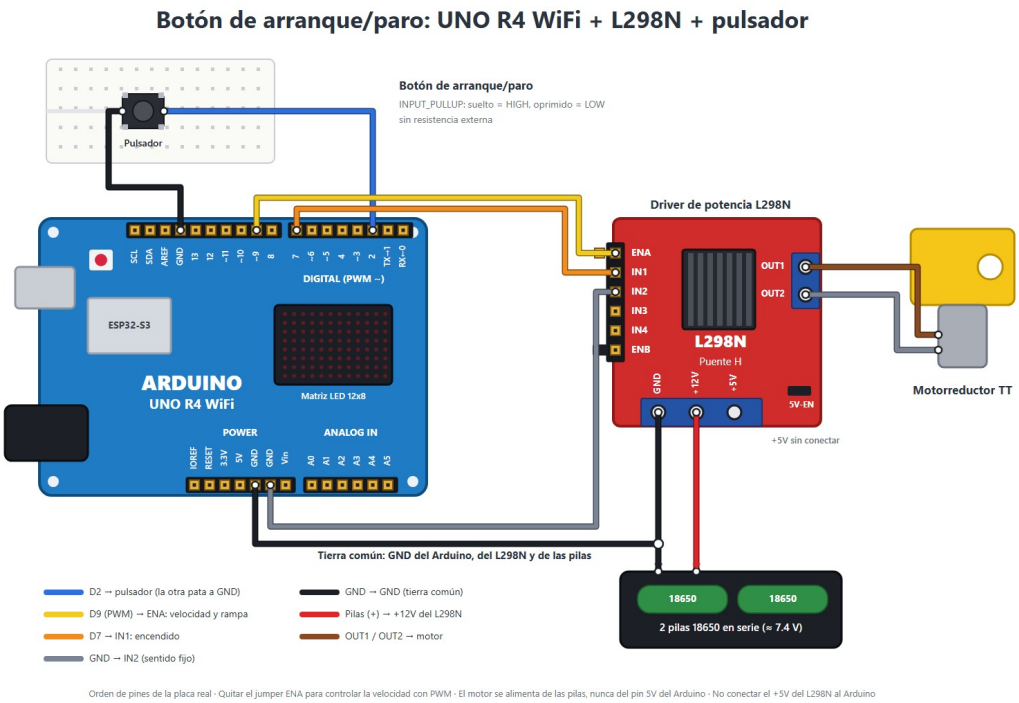


Figura 3. Diagrama de conexión del armado real: UNO R4 WiFi, L298N, motorreductor, pilas 18650 y pulsador.

4. Cálculos y verificación

4.1 Rampa de arranque

Durante la rampa, el programa calcula el PWM con aritmética entera: PWM = 255 · t / 1500, donde t es el tiempo en ms desde la pulsación, medido con `millis()`. La barra de la LCD tiene 16 casillas y se calcula como (PWM · 16 + 255) / 256. La tabla muestra los valores que da el programa en algunos instantes (calculados con esas fórmulas, no medidos).

t (ms)	PWM	% de 255	Casillas de la barra
0	0	0 %	0 de 16
250	42	16 %	3 de 16
500	85	33 %	6 de 16
750	127	50 %	8 de 16
1000	170	67 %	11 de 16
1250	212	83 %	14 de 16
1500	255	100 %	16 de 16

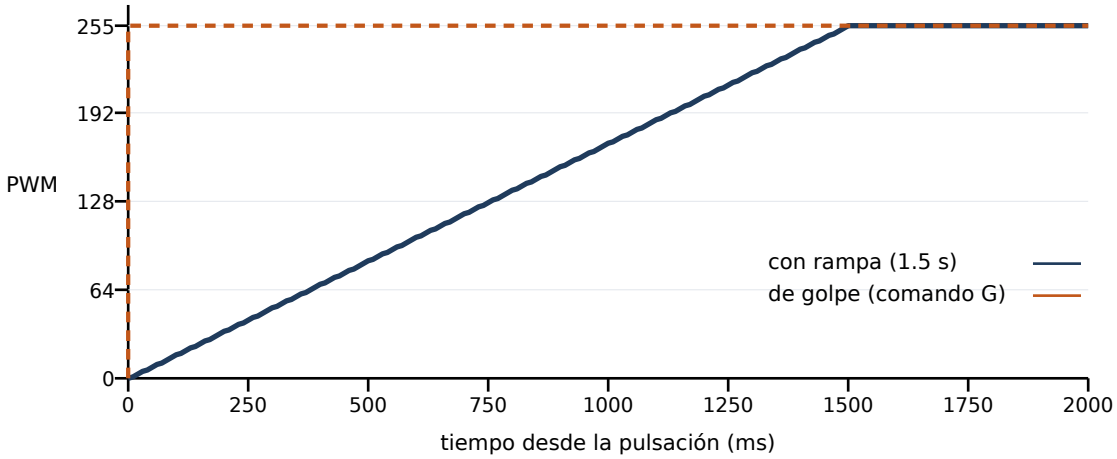


Figura 4. PWM aplicado al motor en el arranque con rampa y en el arranque de golpe.

El PWM sube una unidad cada 1500 / 255 ≈ 5.9 ms, así que el voltaje promedio sobre el motor crece casi de forma continua. En el arranque de golpe el PWM pasa de 0 a 255 en una sola instrucción, y el motor, todavía quieto, pide de una vez todo su pico de corriente. Ese pico es el que puede bajar el voltaje y reiniciar al Arduino en el armado real.

4.2 Ventana del filtro

El filtro solo acepta un cambio si la señal se queda quieta toda la ventana. Los rebotes duran de 1 a 10 ms, así que con 25 ms queda un margen de al menos 15 ms sobre el rebote más largo. El costo es un pequeño retraso: la pulsación se acepta 25 ms después del último rebote, unos 35 ms en el peor caso, que no se percibe al usar el botón.

Ventana	Rebote de 10 ms	Pulsación corta	Resultado
2 ms	Se cuela (10 > 2)	Se cuenta	Esperado: cuenta de más con un botón real
25 ms	Se filtra (10 < 25)	Se cuenta	Observado: cuenta bien (Pulsacion #4)
300 ms	Se filtra	Se ignora si dura menos de 0.3 s	Observado: no contó la pulsación corta

4.3 Velocidad de la simulación

En una de las pruebas Tinkercad avanzó 3.4 s de simulación en 134 s reales: 134 / 3.4 ≈ 39, es decir, corrió unas 39 veces más lento que el tiempo real. A ese ritmo, la rampa de 1.5 s tarda cerca de 1.5 · 39 ≈ 59 s en la pantalla. Esto explica por qué, al intentar 10 pulsaciones seguidas, algunas se perdían por el simulador y no por el filtro.

5. Análisis de resultados

- **Arranque seguro:** al iniciar y después del apagón el motor quedó apagado, con el aviso en la LCD y en el Monitor Serie, y solo arrancó con una pulsación nueva. El sistema no hizo nada que nadie ordenara.
- **Un botón, dos acciones:** cada pulsación aceptada alternó el motor entre arrancado y apagado, y el contador subió de uno en uno (pulsaciones #2, #3 y #4).
- **Rampa y paro:** el arranque pasó por «RAMPA» hasta llegar a PWM 255, mientras que el paro fue inmediato, sin rampa.
- **Ventana del filtro:** con 300 ms se perdió una pulsación corta que con 25 ms sí se contó, lo que confirma que una ventana demasiado larga hace lento al botón.
- **Sin bloqueos:** la LCD, la rampa y los comandos del Monitor Serie respondieron durante toda la prueba, porque el programa no usa `delay()`.

Alcance: el pulsador de Tinkercad es ideal y cambia de estado una sola vez, así que la versión sin filtro cuenta lo mismo que la versión con filtro. Por eso la tabla de rebotes de la práctica queda pendiente de medirse con un botón físico:

Prueba	Pulsaciones reales	Pulsaciones contadas	Veces que el motor no obedeció
Sin filtro, intento 1	10	Pendiente	Pendiente
Sin filtro, intento 2	10	Pendiente	Pendiente
Sin filtro, intento 3	10	Pendiente	Pendiente
Ventana 2 ms	10	Pendiente	Pendiente
Ventana 25 ms	10	Pendiente	Pendiente
Ventana 300 ms	10	Pendiente	Pendiente

6. Preguntas de reflexión

1. Explica con tus palabras qué es el rebote y por qué el Arduino lo detecta aunque tú no lo notes.

Dentro del botón hay una laminita de metal que, al oprimirla, choca contra los contactos y rebota varias veces antes de quedarse pegada, como una pelota que cae al piso. Todo pasa en unos pocos milisegundos (de 1 a 10 ms), así que una persona siente un solo clic. Pero el Arduino lee el pin miles de veces por segundo y alcanza a ver cada separación de la laminita como un cambio de la señal: oprimido, suelto, oprimido, suelto. Para él son varias pulsaciones.

2. ¿Por qué el contador debe sumar solo al pasar de suelto a oprimido y no en cualquier cambio de la señal?

Porque cada pulsación completa tiene dos cambios: uno al oprimir (de HIGH a LOW) y otro al soltar (de LOW a HIGH). Si el contador sumara en cualquier cambio, contaría 2 por cada pulsación aunque no hubiera ningún rebote, y el motor se prendería y se apagaría con un solo clic. Al sumar solo en el paso de suelto a oprimido, una pulsación vale exactamente 1.

3. ¿Por qué el rebote al soltar el botón también puede sumar pulsaciones?

Porque al soltar, la laminita también rebota antes de separarse por completo, y la señal se ve como HIGH, LOW, HIGH. Esa bajada falsa a LOW es un paso de suelto a oprimido, así que un programa sin filtro la cuenta como si el usuario hubiera oprimido otra vez. El filtro lo evita porque ese LOW dura menos de 25 ms y nunca se acepta.

4. ¿Qué problema tendría usar delay(50) para filtrar el rebote en lugar de millis()?

delay() detiene todo el programa mientras espera. Durante esos 50 ms el Arduino no puede subir la rampa del motor, actualizar la pantalla, leer el Monitor Serie ni revisar otras entradas, y con cada pulsación el sistema se «congela» un momento. Además, delay(50) no filtra de verdad: solo espera un tiempo fijo después del primer cambio, sin comprobar que la señal se haya quedado quieta, así que un rebote que llegue después de la espera, por ejemplo al soltar, se sigue contando. Con millis() el programa solo anota la hora y sigue trabajando, y acepta el cambio cuando la señal lleva el tiempo completo sin moverse.

5. Según tus pruebas, ¿qué pasa si la ventana de estabilidad es muy corta? ¿Y si es muy larga?

Con una ventana muy larga (300 ms) se pierden las pulsaciones rápidas: en la simulación, una pulsación corta no se contó con 300 ms y la misma pulsación sí se contó con 25 ms. Además, el botón se siente lento porque el sistema tarda en reaccionar. El efecto de una ventana muy corta (2 ms) no se pudo medir en Tinkercad, porque su botón no rebota; con un botón real se esperaría que se colaran rebotes, ya que duran de 1 a 10 ms y muchos superan los 2 ms. Por eso 25 ms es un buen punto medio: es más largo que cualquier rebote y mucho más corto que una pulsación humana.

6. ¿Por qué es peligroso que un sistema reinicie con el motor encendido? Da un ejemplo industrial.

Porque al volver la energía, la máquina podría arrancar sola sin que nadie lo ordenara, justo cuando alguien la está revisando o tiene las manos cerca. Por ejemplo, en una banda transportadora, una sierra o una prensa, un operador puede acercarse a destrabar una pieza después de un parpadeo de luz; si la máquina se reanuda sola, lo puede lastimar. Por eso las máquinas industriales exigen oprimir de nuevo el botón de arranque después de un corte.

7. ¿Por qué la rampa de arranque ayuda a evitar caídas de voltaje?

Porque un motor quieto pide mucha más corriente al arrancar que cuando ya gira: todavía no genera la fuerza contraelectromotriz que limita la corriente. Si el PWM pasa de 0 a 255 de golpe, el motor jala ese pico de una vez, la fuente no lo alcanza a dar y el voltaje baja, lo que puede reiniciar al Arduino. Con la rampa, el voltaje promedio sobre el motor sube poco a poco, el motor gana velocidad mientras la corriente sube y el pico es mucho menor.

8. Menciona dos medidas de hardware que ayudan a evitar caídas de voltaje y que el software no puede reemplazar.

Primera: alimentar el motor con una fuente separada y suficiente, como las pilas 18650 o una fuente con la corriente adecuada, nunca desde el pin 5V del Arduino, y con las tierras unidas. Segunda: poner un capacitor electrolítico (por ejemplo, de 470 a 1000 µF) cerca de la alimentación del driver, que entrega la corriente del pico de arranque y amortigua la caída. También ayudan los cables gruesos y cortos y las conexiones firmes, porque los cables delgados o flojos y los malos contactos de la protoboard provocan caídas de voltaje que ningún programa puede corregir.

7. Conclusión

Los resultados coinciden con el diseño: el sistema arrancó siempre en estado seguro, incluso después del apagón, y el motor solo se movió con una pulsación nueva. La rampa llevó el PWM de 0 a 255 en 1.5 s, el paro fue inmediato y la prueba de la ventana mostró que 25 ms cuenta bien las pulsaciones, mientras que 300 ms pierde las rápidas.

La simulación permitió comprobar todo el comportamiento del programa, pero también mostró sus límites: el botón de Tinkercad no rebota y el simulador corre unas 39 veces más lento con la LCD, así que la medición del rebote necesita un botón físico. El software protege al sistema, pero no reemplaza una buena alimentación ni buenas conexiones.